

## 1. Inserting a Key in the Hash Table and Solving Overflow

Method for Inserting a Key:

Index Calculation: The key is hashed to determine its index in the hash table using the formula  $\text{key} \& ((1 \ll \text{global\_depth}) - 1)$ .

Bucket Selection: The bucket at the calculated index is selected for insertion.

Overflow Handling: If the selected bucket is full (i.e.,  $\text{bucket} \rightarrow \text{num\_entries} \geq \text{bucket\_size}$ ), the extend method is called to handle overflow.

Solving Overflow:

Extend Method:

The local depth of the bucket is increased.

If the local depth exceeds the global depth, the global depth is also increased, and the table size is doubled.

A new bucket is created, and entries are redistributed between the original and new buckets based on their hash keys.

This redistribution ensures that the table can accommodate more entries without immediate overflow.

## 2. Removing Keys and Shrinking the Bucket

Method for Removing Keys:

Index Calculation: Similar to insertion, the key's index is calculated.

Bucket Traversal: The bucket at the calculated index is traversed to find the entry with the matching key.

Entry Removal: The entry is removed by adjusting pointers, and the shrink method is called to check if the bucket can be merged with its pair.

Shrinking the Bucket:

Shrink Method:

Checks if the bucket's local depth is greater than 1.

Identifies the pair bucket using the XOR operation on the hash key.

If both buckets have the same local depth and one is empty, they are merged.

The local depth of the non-empty bucket is decreased, and the table is resized if possible.

### 3. B+ Trees vs. Extendible Hashing

Basic Idea of a B+ Tree:

A B+ tree is a self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time.

It is a type of B-tree where all values are stored at the leaf level, and internal nodes only store keys to guide the search.

Differences Between B+ Trees and Extendible Hashing:

Structure: B+ trees are hierarchical, while extendible hashing uses a flat structure with buckets.

Data Access: B+ trees allow for range queries and ordered data access, whereas extendible hashing is optimized for exact match queries.

Space Utilization: B+ trees can be more space-efficient as they do not require doubling the table size like extendible hashing.

Which is Better?

Use Case Dependent: The choice between B+ trees and extendible hashing depends on the specific use case.

If you need ordered data access and range queries, B+ trees are preferable.

If you need fast exact match queries and dynamic resizing, extendible hashing might be better.

Feel free to express your own ideas or ask for further clarification on any of these points!